



Cybersecurity & Penetration Testing Lab Environment Setup

Building an isolated virtual lab for multi-target penetration testing, ethical hacking practice, and security research.



Project Overview

This project focuses on building a dedicated, multi-OS penetration testing laboratory using **VMware Workstation**. The primary goal is to establish an isolated `10.0.0.0/24` network environment containing an attack platform (**Kali Linux**) interconnected with multiple target machines spanning **Windows** (Win 7, Win 10, Win 11) and **Android** environments.

The lab is configured on a custom isolated virtual NAT network (`vmnet8`), enabling controlled vulnerability assessments, service enumeration, exploitation, and post-exploitation testing without exposing local physical networks or unauthorized systems to risk.



Objectives

- Configure custom virtual networking (`vmnet8` NAT) in VMware Workstation.
- Deploy and configure **Kali Linux** as the primary attack platform on `10.0.0.2`.
- Provision and network multiple target operating systems across specific IP allocations:
 - **Android-x86 Target:** `10.0.0.9`
 - **Windows 10 Target:** `10.0.0.10`
 - **Windows 11 Target:** `10.0.0.16`
 - **Windows 7 Target:** `10.0.0.7`
- Establish and verify full local network inter-VM routing and internet access (`8.8.8.8`).
- Configure custom host firewalls (ICMP ingress rules) and static IP parameters on targets.
- Establish baseline VM state recovery mechanisms.



Purpose of the Lab

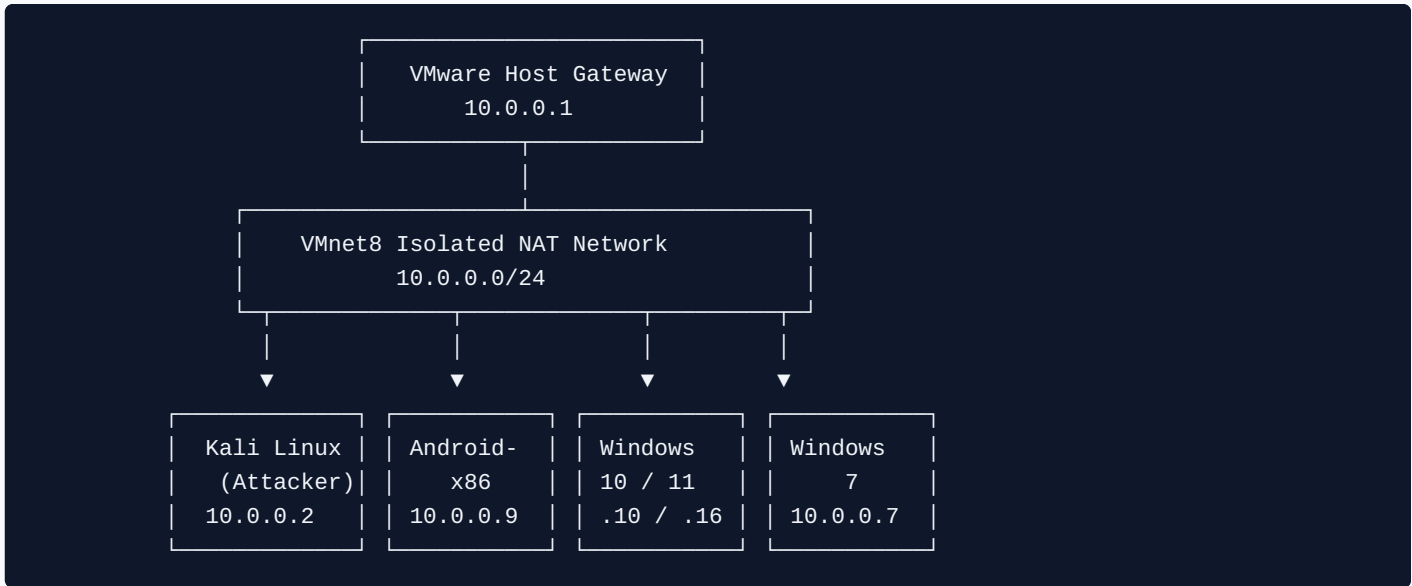
This lab provides a safe, repeatable, and completely contained ecosystem for authorized hands-on security training. Activities designed for this environment include:

- **Network Reconnaissance & Port Scanning:** Service identification across Windows and Android platforms.
- **Android Security Auditing:** ADB (Android Debug Bridge) exploitation, application assessment, and shell access.
- **Windows Exploitation Practice:** SMB, RPC, and OS-specific vulnerability assessments.
- **Packet Analysis & Traffic Inspection:** Capturing virtual interface traffic using Wireshark.
- **Exploitation & Payload Testing:** Metasploit Framework and custom exploit payload delivery.



Disclaimer: This environment and its associated documentation are intended strictly for educational and authorized security testing purposes. All testing must be confined to systems owned or explicitly authorized by the lab operator.

🏗️ Lab Architecture



⚙️ Lab Configuration

Component	Configuration	Description
Hypervisor	VMware Workstation Pro / Player	Virtual machine management and network emulation
Virtual Network	VMnet8 (NAT)	Subnet: 10.0.0.0/24
Default Gateway	10.0.0.1	Assigned to VMware host adapter (VMnet8)
DNS Server	8.8.8.8	External resolution forwarded through NAT gateway
Attacker VM	Kali Linux (10.0.0.2)	OS Kernel 6.x, Dual-NIC config (eth0 active)
Target 1 (Android)	Android-x86 (10.0.0.9)	Kernel: Other Linux 5.x 64-bit
Target 2 (Windows 10)	Windows 10 (10.0.0.10)	Static IPv4, Custom ICMP Ingress enabled
Target 3 (Windows 11)	Windows 11 (10.0.0.16)	Enterprise/Pro Evaluation instance
Target 4 (Windows 7)	Windows 7 (10.0.0.7)	Legacy OS instance for vulnerability research

📋 Lab Setup Procedure

Step 1. Configure VMware Virtual Network Editor (VMnet8)

1. Opened VMware **Virtual Network Editor**.
2. Configured **VMnet8** as a **NAT** network type.
3. Set Subnet IP to 10.0.0.0 and Subnet Mask to 255.255.255.0 .
4. Set Gateway IP inside NAT Settings to 10.0.0.1 .

[IMAGE: Virtual Network Editor Configuration - Screenshot 2026-09-10 215300.png]

Step 2. Configure Host Network Adapter (`VMnet8`)

1. Opened **Network Connections** (`ncpa.cpl`) on the Windows host.
2. Configured **VMware Network Adapter VMnet8** IPv4 properties with static IP `10.0.0.1` , subnet mask `255.255.255.0` , and DNS server `8.8.8.8` .

[IMAGE: Host IPv4 Static Configuration - Screenshot 2026-09-10 230239.png]

Step 3. Deploy and Network Kali Linux (Attacker)

1. Imported Kali Linux pre-built image into VMware Workstation.
2. Attached Network Adapter 1 to **Custom: VMnet8 (NAT)**.
3. Set static IPv4 configuration on interface `eth0` via NetworkManager:
 - **IP Address:** `10.0.0.2`
 - **Netmask:** `24` (`255.255.255.0`)
 - **Gateway:** `10.0.0.1`
 - **DNS:** `8.8.8.8`

[IMAGE: Kali Linux IPv4 Connection Settings - Screenshot 2026-09-10 222034.png]

Step 4. Provision Windows Targets

1. Deployed Windows 10 (`10.0.0.10`), Windows 11 (`10.0.0.16`), and Windows 7 (`10.0.0.7`) instances on `VMnet8` .
2. Assigned static network parameters via `ncpa.cpl` .
3. Configured Windows Firewall to respond to ICMP Echo Requests (Ping):

```
netsh advfirewall firewall add rule name="Allow ICMPv4" protocol=icmpv4:8,any dir=in action=allow
```

Step 5. Install and Network Android-x86 Target

1. Created new VM shell: Guest OS set to **Linux > Other Linux 5.x kernel 64-bit**.
2. Mounted `android-x86_64-9.0-r2.iso` and completed partitioning (`ext4` , GRUB bootloader).
3. Configured networking on interface `eth0` to `10.0.0.9/24` with Gateway `10.0.0.1` .
4. Unmounted ISO image to ensure proper hard disk boot.

Lab Verification

Verified full network visibility and reachability from the Kali Linux terminal (`10.0.0.2`) across local interfaces, host gateway (`10.0.0.1`), and external DNS (`8.8.8.8`).

[IMAGE: Kali Terminal Ping Verification - Screenshot 2026-09-10 225353.jpg]

Test	Command / Action	Expected Result	Status
Check Kali Interface	<code>ip a</code>	<code>eth0</code> assigned <code>10.0.0.2/24</code>	✓ PASS
Local Gateway Reachability	<code>ping -c 4 10.0.0.1</code>	<code>0% packet loss</code>	✓ PASS
External Connectivity	<code>ping -c 4 8.8.8.8</code>	<code>0% packet loss</code> (Internet active)	✓ PASS
Windows 10 Target Ping	<code>ping -c 4 10.0.0.10</code>	ICMP reply from Windows target	✓ PASS
Android Target Ping	<code>ping -c 4 10.0.0.9</code>	ICMP reply from Android target	✓ PASS



Problems Encountered & Solutions

Problem 1: Android VM Booting into Console Prompt (`console:/ #`)

- **Symptom:** After running the installer, restarting the Android VM resulted in a black screen displaying `console:/ #` or `Detecting Android-x86... found at /dev/sr0` instead of booting into the graphical Android UI.
- **Cause:** VMware was still forcing boot from the attached Android installer ISO mounted on the virtual CD/DVD drive.
- **Solution:** Navigated to **VM Settings > CD/DVD (IDE)**, unchecked **Connected** and **Connect at power on**, unmounted the ISO, and forced a VM restart.

Problem 2: Target Windows VMs Dropping Ping Packets

- **Symptom:** Running `ping 10.0.0.10` from Kali resulted in `100% packet loss`, despite network adapters being correctly assigned to `VMnet8`.
- **Cause:** Default Windows Defender Firewall rules block incoming ICMPv4 Echo Requests.
- **Solution:** Executed administrative command prompt rule addition in Windows to explicitly allow ICMP ingress traffic.



Key Learnings

1. **VMware Networking Architecture:** Mastered difference between Host-Only (`VMnet1`), NAT (`VMnet8`), and Bridged networking models for controlled security lab design.
2. **Multi-OS Administration:** Gained practical experience configuring static IP stacks across Linux, Windows, and Android kernel environments.
3. **Target Hardening Awareness:** Identified how default OS firewall policies affect network scanning visibility and enumeration.
4. **Virtualization Troubleshooting:** Learned ISO detachment procedures, partition management (`ext4` /GRUB), and display parameters required for non-standard guest operating systems like Android-x86.